

AEGIS

Governed-Agent Control Plane for Amazon Bedrock AgentCore

PARTNER BRIEF

Prepared	10 September 2026
Verified against	v0.7.0-pilot-rc1 — benefits_eligibility_agent, tag == main
Live result	21 / 21 full-portfolio gate, from zero, torn down to zero residue
Scope of that result	One pack. One AWS account. One region (us-east-1). Two tenants. Synthetic data.
Attempts to get there	Seven for the gate itself, and a further six for the system-of-record connector alone. Every one of those failures is published in the repository beside the pass.
Independently verified	No. That is the specific thing this brief asks a partner to change.
Governing document	NOT - CLAIMS .md — if any wording here reads as stronger, that page wins.

How to read this document

This brief is written to be checked, not believed. Every claim is followed by the file, test or run that demonstrates it, or by a plain statement that nothing does. Section 4 is the list of things that are *not* proven; it is longer than section 3, and that is deliberate.

1. The one-paragraph version

Aegis is a **deny-by-default governance control plane for LLM agents on AWS**. It places Cedar authorization, PII masking proven by signature, per-tenant cost ceilings, a kill switch and a WORM audit ledger *in front of* the model rather than inside the prompt — so the governance cannot be talked out of by the agent it governs. It has been deployed from zero and torn down repeatedly on a real AWS account, most recently passing a 21-check full-portfolio gate with a live AgentCore Runtime on 10 September 2026 — the four new checks being a governed outbound connector to an OAuth2 system of record, and the sweep that proves it left nothing behind.

It is a reference architecture, not a product. There is no SLA, no support contract and no managed release train. And every piece of evidence in it was produced by its author, on the author's own account — which the repository itself names as *“the single biggest credibility gap in the project.”*

The ask, in one sentence

That gap is the reason to talk to a partner, and closing it is the only thing we are asking a partner to do first.

2. Where this platform actually is today

An honest self-assessment, written against the bar a public-sector or healthcare customer would actually apply: can this give an auditor what they require for LLMs in production on AWS?

2.1 What is genuinely strong

- **The enforcement point is in the right place.** Twelve forbid policies and one permit, evaluated by AWS's GA Cedar engine at the AgentCore Gateway — not in a system prompt, not in application code the agent could reason around. Cedar is forbid-wins, so the twelve are load-bearing and the permit is only the door.
- **Masking is proven, not asserted.** A caller-supplied deidentified: true is never accepted. The proof is a signed sanitized_ref, and a forged reference is refused. This is the strongest single control in the system, and it is what makes the four mask_before_* policies mean anything.
- **The workflow cannot be talked out of a control.** A deterministic Step Functions STANDARD machine runs a guard after every substantive step and cannot advance on unverified state. The model is a participant, never the controller.
- **It is genuinely configurable.** lib/engine/render.py turns an agent manifest into gateway targets, Cedar policy statements and guardrail configuration. Twenty-seven CDK context switches cover network mode, KMS mode, retention profile, identity mode, tenancy, perimeter, account-wide capture, model logging and the spend ceiling. Four vertical packs share one hash-locked core, which is evidence that it ports.
- **The honesty apparatus is unusual, and it is an asset.** Seventy recorded findings, including every one where the author was wrong. A NOT-CLAIMS page that governs every other page. Failed runs published beside passing ones.

2.2 Where it is weak — read this before the technical conversation

The first version of this section led with observability. That judgement rested on a number I measured wrongly, and the correction is the first thing below — both because it changes the conclusion and because how a team handles its own bad number is worth more signal than the number was.

Correction — an earlier draft of this brief got this number badly wrong

The 9 September draft said this platform deploys “**three alarms**” and called observability its thinnest layer. That came from counting `cw.Alarm()` call sites in the source; two of the three are inside loops. The synthesized CloudFormation template carried **twenty-two**. The error was the method, not the arithmetic: counting the generating code instead of the deployed artifact.

It is disclosed here rather than quietly fixed because a partner is entitled to know which way a number moved, and because the same defect appears twice more in this repository's own register (L60, L67). The count now comes from `tools/observability_census.py`, which reads the template, and its `--assert-baseline` mode fails if these figures drift from what this document says.

Observability after the OBS-1..OBS-4 and OBS-6 uplift

26 alarms across workflow health, per-function errors, guard failures, per-tenant budget at three thresholds, latency and perimeter bypass — **every one of them named** (nine previously existed only as CloudFormation logical ids, so no runbook could reference them and no operator could find them in the console).

Three severity tiers as composite alarms, each with its own SNS topic: P1 page, P2 ticket, P3 advisory. Latency is now alarmed rather than merely charted. A stalled human sign-off is detected in **two hours instead of twenty-four**. And every alarm has an entry in `docs/ops/INCIDENT-RUNBOOK.md` — enforced by a test, so a new alarm without a documented first action fails CI.

Still open, and named: per-tenant dashboards, a synthetic canary, proving each tier actually fires in a live incident, and a DR exercise. See `docs/ops/OBSERVABILITY-UPLIFT.md`.

- **Account-wide prevention does not exist.** There is no AWS Organization, therefore no SCP. A principal calling Bedrock directly is detected, not prevented. For public sector this is frequently the question that decides the deal, and today the answer is a lint-validated template that has never been tested in a real OU.
- **COMPLIANCE-mode immutability is proven at one day, not seven years.** The mechanism is proven: `BypassGovernanceRetention` is refused and the deployment's own teardown fails. The duration is a parameter. Stating it any other way invites an auditor to find the gap themselves.
- **The thirteen Cedar policies are a worked example, not a library.** They name benefits-specific tool actions. A customer authors their own against their own manifest. That is still a strong story — but “configurable governance platform” and “here are thirteen reusable policies” are different claims.
- **Three of the four packs have never been deployed.** Same hash-locked core, propagated fixes, unit tests, CI parity on fifteen shared files — and zero from-zero deployments. Housing wires 3 of 17 controls.
- **No CUSTOMER system of record, and no real data, ever.** A governed OAuth2 connector IS now deployed and IS a Gateway target, and it is proven live: the tool holds no client secret, the token is minted by the AgentCore Identity vault, and the system of record verifies its RS256 signature against the issuer's live JWKS. But that system of record is ours. `connect_system_of_record` — the one that would reach a state benefits system — remains stubbed. Every live run governed synthetic data.

2.3 The recommendation**Ready for this conversation. Not ready to be called a product.**

This is a credible governance control plane at reference-architecture maturity, with an unusually well-documented gap list, and it is further along than most teams will have built themselves. Any conversation that lets a partner believe it is a product will fail inside an hour — so the brief leads with what is missing.

The correct next step is narrow, cheap and falsifiable: **the partner's SA runs the independent verification protocol in a clean account**. It is scripted, it takes about half a day, and it either reproduces our results or it does not.

What we are explicitly not asking for yet

Do not ask a partner to find a customer at this stage. That would be asking them to put their name on synthetic, author-produced evidence. The sequence that works is: independent verification, then one real read-only system-of-record connector, then a named customer for a supervised pilot on de-identified data. Skipping to the third step is the single most likely way to end the relationship badly.

3. What is proven

One tree — `benefits_eligibility_agent` at tag **v0.7.0-pilot-rc1** — passed a 21/21 full-portfolio gate on 2026-09-10 (run `ben-fpg`) from an empty environment, with two tenants and a real AgentCore Runtime, then tore down to zero residue. Evidence: `evidence/FULL-PORTFOLIO-GATE-benefits_runtime_agent.json`.

3.1 The twenty-one checks, and what each one actually measured

Check	Result recorded in the evidence file
preflight_no_agentcore_residue	Gateways, policy engines and agent runtimes all empty before deploy. Refuses to deploy otherwise.
deploy	rc=0 in 722.2s, nine CloudFormation stacks.
outputs_present	identity, gateway, compute_runtime_role=ben-fpg-agentcore-runtime.
RT2_configure_on_iac_role	Runtime configured on the CDK-declared execution role, not the toolkit's generated one.
RT2_launch	Runtime launched.
RT2_runtime_uses_iac_role	runtime=benefits_runtime_agent-ccFPa76fqX on role ben-fpg-agentcore-runtime.
RT2_runtime_ready	status=READY.
RT2_runtime_mmdsv2	requireMMDSV2=True — closes the IMDSv1 credential-theft path.
RT2_runtime_calls_guardrail_assessed	runtime_rows=20, guardrail_assessed=20 — every runtime model call was guardrail-assessed.
G111_consolidated_gate	rc=0 in 531.9s. Multi-tenant isolation, per-tenant audit routing 12/12, transparency 13/13 per tenant, strict PII canary clean.
KS_kill_switch_proof	rc=0 in 105.0s. Engaged, blocked, disengaged — IAM-attributed and WORM-audited.
BUD_budget_proof	rc=0 in 373.3s. Per-tenant token and USD ceiling refused the spend before it happened.
LIN_case_driven	Case LIN-0E6D3D driven through all ten workflow states to HumanSignoff.
LIN_zero_orphans	orphans=0, invokes=11, aegis=11. CloudTrail invocations reconciled against audit rows in both directions.
E2E_zero_unexpected	rc=0. Remaining errors are the deliberate fail-closed refusals, correctly classified.
CONN_deploy	NEW. rc=0(ok=True) outbound_ok=True artifacts_ok=True in 106.7s. The governed connector stood up AND its outbound leg was exercised against the running system of record during the deploy. Two earlier runs passed the old artifacts-only version of this check over a connector whose system of record trusted nothing.
CONN_governed_sor_proof	NEW. rc=0, 4 of 4. The system of record refuses no-token and bad-token with 401; the tool holds no client secret; the outbound token is minted by AgentCore Identity; and an outsider calling <code>verify_source</code> is DENIED by Cedar — with the denial naming a specific policy, not refusing everyone alike.
CONN_tearardown_zero_resid	NEW. Proof users, resource server, hosted domain, credential provider, Lambdas,

Check	Result recorded in the evidence file
ue	IAM role and HTTP API all removed.
teardown_zero_stack_residue	stacks_clean=True. destroy_rc=1 by design — the residue sweep is the claim, the CLI exit code is not (register L28).
teardown_zero_agentcore_residue	Gateways, policy engines and runtimes all empty after teardown.
teardown_zero_connector_residue	NEW. left=[], unchecked=[]. Asks AWS directly for each connector resource and counts “could not check” as a failure — it does not consult the teardown script’s own verdict, because that verdict is the thing under suspicion (L70).

3.2 Newly proven since the previous release

- **Policy provenance, in a live audit row.** The run wrote hash-chained ledger rows carrying policy_version = cedar-534be1c676bb and rule_version = manifest-835adf581b0c into ben-fp2-sp-a-audit-ledger. An auditor asking “which policy decided this case?” now gets an answer from the record. Previously this was configuration-asserted only.
- **Zero AgentCore residue.** Gateways, policy engines and agent runtimes are AWS resources that CloudFormation teardown does not own, so the previous “zero residue” claim covered stacks only. It is now asserted before deploy and after teardown.
- **A governed system-of-record connector, proven end to end (CONN-1).** verify_source is a Gateway target that authenticates to an OAuth2 system of record using a machine-to-machine token from the AgentCore Identity vault, and holds no client secret itself. The system of record verifies the token’s RS256 signature against the issuer’s live JWKS and answers an unauthenticated caller with 401. The system of record is real — and it is ours; connect_system_of_record stays stubbed for exactly that reason. It took six from-zero live runs to get right, and not one of the six failures was where it appeared to be.
- **COMPLIANCE-mode Object Lock (EV-1).** Proven at one-day retention: BypassGovernanceRetention is refused and the deployment’s own teardown path fails. The mechanism is proven; the seven-year duration is a parameter.

3.3 It took seven attempts for the gate, and six more for the connector — all published

evidence/FULL-PORTFOLIO-GATE-2026-09-08.json is the run immediately before the pass. It reached the checks and failed four of them. It sits in the same commit as the pass. **If a partner SA reads one thing to decide whether this evidence is honest, it should be those two files side by side.**

#	What went wrong	Register
1	Line-ending noise that was cosmetic — and which the author first diagnosed as digest corruption and was wrong . Tested before publishing; the claim was false and the commit message was rewritten.	L60
2	Three consecutive wrong diagnoses of one ConflictException. The real cause: AgentCore policy names are unique per account and region, not per policy engine , so a leftover engine from a sibling pack held the names. lib/engine/render.py’s own comment had said so the whole time. All four packs now prefix policy names.	L64
3	The CloudTrail five-trails-per-region quota, which AWS marks not adjustable. A service limit, not a defect. Still open; a support case is drafted.	L61
4	RT-4 — a control that worked and made the agent unreachable. See section 4.	L65, L66

4. What is not proven

These are not hedges. Each is a specific thing a customer's auditor will ask about, and the answer today is written here rather than discovered later.

#	Not proven	What that means in practice
1	All evidence is author-produced, on one AWS account.	No third party has run it. docs/INDEPENDENT-VERIFICATION.md exists precisely because an earlier reviewer discounted the evidence on these grounds. No amount of further self-testing closes this.
2	No CUSTOMER system of record has ever been governed.	This changed on 2026-09-10, and the change is narrower than it sounds. verify_source is now a Gateway target, proven live: no client secret in the tool, an outbound OAuth2 token from the AgentCore Identity vault, RS256/JWKS signature verification at the system of record, and a 401 for an unauthenticated caller. But the system of record is OURS — a real OAuth2 API we built and deploy. Proving the governed path reaches it is not proving a customer's system of record has been governed, so connect_system_of_record remains stubbed in the manifest. Every live run still governed synthetic data.
3	Account-wide prevention is absent.	No AWS Organization, therefore no SCP. A non-governed principal calling Bedrock directly is detected, not prevented. The SCP is a lint-validated template, never tested in a real OU (PERIM-1b).
4	R4-2 is OPEN and was reopened deliberately.	RT-4 restricted the runtime authorizer to the Gateway workload. It works — and the Gateway is downstream of the Runtime here, so it left no permitted invoker and failed four gate checks. RT-4 is now opt-in and off by default. The bypass is mitigated at the gateway by the four mask_before_* forbids, which refuse the consequential actions unless the payload is proven de-identified, whoever calls.
5	Operations posture is pilot-grade — though better than an earlier draft claimed.	26 named alarms in 3 severity tiers, latency alarmed, approval backlog detected in 2h, and a runbook entry for every alarm enforced by CI. Still absent: per-tenant operational dashboards, a synthetic canary, any evidence that a tier has ever fired in a real incident , a real on-call rotation, and a DR exercise. An SLO alarm is a threshold we chose; it is not an SLO agreed with anyone.
6	Three of four packs are unproven live.	Pharmacovigilance, financial-aid and housing share the hash-locked core and receive every fix with unit tests. None has ever been deployed. Housing wires 3 of 17 controls.
7	COMPLIANCE retention proven at 1 day, not 7 years.	Mechanism proven, duration is a parameter.
8	Enterprise IdP federation round-trip.	Configuration exists; the round-trip has never been exercised.
9	No penetration test, DR exercise, SLO or ATO.	None has been performed. The platform carries no authorization of any kind.
10	488 checkov findings are baselined portfolio-wide.	Lambda concurrency limits, DLQs, VPC attachment, log-group and env KMS, DynamoDB CMK and PITR. The baseline is reviewed and gated so new findings fail CI, but the residual is real.
11	“Zero residue” excludes ECR.	Teardown does not prune the toolkit repository, which grows one image per run (L37). Cost is negligible; the claim was the problem.
12	No scale, multi-region or multi-	Multi-account aggregation to a log-archive account is open (PERIM-6).

#	Not proven	What that means in practice
	account evidence.	

5. The Cedar policy set

Thirteen policies live in `policies/*.cedar` and are attached to the AgentCore Cedar Policy engine in ENFORCE mode by a CloudFormation custom resource at deploy time. **Twelve are forbid. One is permit.**

Why that asymmetry is the whole architecture

Cedar is **forbid-wins**: any single forbid that matches kills the request no matter how many permits fire. So the twelve forbids are the product, and the permit is only the door being unlocked. A caller with a missing or malformed claim is granted **zero** tools, not all of them — the failure mode most agent stacks get backwards.

Optional context fields are presence-guarded, so an absent value fails **closed**: the forbid fires. The GA engine rejects an unguarded optional-attribute access outright, which is why every policy below reads “context.input has X && ...”.

5.1 How a policy is applied

1. The agent manifest declares the tool targets. `lib/engine/render.py` emits `policies.tsv`, substituting the gateway ARN once the gateway exists.
2. The gateway REQUEST interceptor sets the authoritative context — `tenant`, `budget_ok`, `within_service_window` — from live sources. Any value the caller supplied for these is overwritten.
3. The Cedar engine evaluates in ENFORCE. Forbid wins. The decision is made by AWS, before the tool Lambda is invoked.
4. The decision, with `policy_version` and `rule_version`, is written to the hash-chained audit ledger before the side effect occurs.

5.2 The thirteen policies

Group A — Admission: is this caller allowed to do anything at all?

Policy	Kind	What it does, and what it stops
<code>require_tenant</code>	forbid	A principal whose identity carries no tenant may call no governed tool at all. The tenant is derived from the caller's own validated JWT — never from the request — and the target routes to that tenant's store. Cross-tenant access is impossible by construction, not by check. Attached only in multi-tenant deployments.
<code>require_entitlement</code>	forbid	Zero-default tools. Being in the caseworker role group is not enough ; the principal needs either a non-empty <code>custom:tools</code> claim or membership of <code>tools_granted</code> . With neither, tools are zero. Defence in depth for the interceptor, which already drops the advertised tool list.
<code>require_service_window</code>	forbid	No governed decision action outside the deployment's service window. <code>within_service_window</code> is set by the interceptor from the real request time and is authoritative — any caller value is overwritten. Scoped to the four consequential actions, because the built-in AgentCore actions carry no input in their context.

Group B — De-identification: nothing consequential touches un-masked data

Policy	Kind	What it does, and what it stops
<code>mask_before_assess</code>	forbid	No eligibility assessment on un-masked data.

Policy	Kind	What it does, and what it stops
<code>mask_before_draft</code>	forbid	No notice drafting on un-masked data — so the model only ever sees de-identified case text.
<code>mask_before_redetermine</code>	forbid	No changed-circumstances re-determination on un-masked data.
<code>mask_before_overpayment</code>	forbid	No overpayment calculation on un-masked data.
The control underneath these four is the strongest thing in the platform Each policy tests <code>deidentified == true</code>, which is set only by <code>mask_pii</code>. But the boolean is never accepted downstream as proof. The authoritative control is a cryptographically signed sanitized_ref: a forged or tampered reference is refused by the guard, and the raw application text — and the drafted notice — never cross workflow state at all (pass-by-reference). This is also why reverting RT-4 is defensible rather than a retreat. Even if a caller reaches the runtime directly and bypasses the gateway interceptor, the consequential actions still require the signature. RT-4 would have closed a path; these four close the action. That is the stronger place to enforce, and it is live-proven rather than unit-tested.		

Group C — Separation of duties: what the agent may never do

Policy	Kind	What it does, and what it stops
<code>no_self_commit</code>	forbid	The agent can never commit a determination. <code>finalize_determination</code> is forbidden outright and hidden from the advertised tool list . The only path to a commit is the human sign-off gate, where a different qualified caseworker approves with a single-use token.
<code>no_self_fraud_referral</code>	forbid	A fraud referral is a consequential, human-only action. <code>refer_fraud</code> is forbidden outright and hidden. Same pattern as <code>no_self_commit</code> — every new high-risk action gets its own forbid.

Group D — Purpose, spend and quantitative limits

Policy	Kind	What it does, and what it stops
<code>consent_purpose_before_assess</code>	forbid	Purpose limitation. <code>assess_eligibility</code> is refused unless recorded consent is present and the declared purpose is one of eligibility or redetermination. A coarse gateway gate layered on the authoritative server-side controls — no single boolean is accepted downstream as proof.
<code>budget_before_draft</code>	forbid	<code>draft_notice</code> is the only tool that spends model tokens, so it carries the spend gate. <code>budget_ok</code> is set by the interceptor from the live per-tenant meter , not asserted by the caller. Refuses <i>before</i> the spend.
<code>amount_cap_overpayment</code>	forbid	An overpayment computation whose <code>prior_monthly_benefit</code> exceeds the caseworker cap (\$5,000) is refused at the gateway — a large-dollar case needs elevated authorization, not a routine call. Uses the Cedar decimal extension, presence-guarded so a missing amount fails closed.

Group E — The single grant

Policy	Kind	What it does
<code>caseworker_permit</code>	permit	A benefits caseworker may use the governed tools. Deny-by-default is automatic at the engine; this is the only broad grant in the set, and it is

Policy	Kind	What it does
		authorized by the Cognito group claim surfaced as a tag. No group, no access — and every forbid above still wins over it.

The honest limit of this policy set

These thirteen are a **worked example, not a reusable library**. They name benefits-specific tool actions such as `assess-eligibility__assess_eligibility`. A customer authors their own against their own manifest — which the render pipeline supports directly, and which is the real platform capability. But “configurable governance platform” and “thirteen reusable policies” are different claims, and only the first is true.

6. Architecture

The as-deployed architecture is attached as **AEGIS-ARCHITECTURE-VERIFIED-2026-09-10.drawio**. It is drawn in eight layers, and every component carries a verification status: **LIVE** (exercised in the 21/21 gate), **IAC** (built by CDK and unit-asserted only) or **OFF** (shipped but deliberately not deployed). A 24-row legend gives every component's purpose and what it stops.

Layer	What it contributes to the governance story
1 — Identity & admission	Cognito issues the token carrying cognito:groups, custom:tenant and custom:tools. Tenancy and entitlement come from the verified token, never the request body. WAF on the front door is IaC-asserted.
2 — Authorization	AgentCore Gateway (MCP, CUSTOM_JWT) plus the GA Cedar engine in ENFORCE, plus the REQUEST interceptor that sets authoritative context. This is the enforcement point, and it is AWS-evaluated.
3 — Agent runtime & model	AgentCore Runtime on the CDK-declared execution role with MMDSv2 required; Bedrock over PrivateLink; a Guardrail whose exact ARN is an IAM StringEquals condition with an explicit Deny on anything else.
4 — Deterministic orchestration	Step Functions STANDARD, ten states, a guard after every substantive step. CheckAdverseNotice is Goldberg v. Kelly in code. AuditIntent writes before the side effect. HumanSignoff is waitForTaskToken with the approver re-verified and required to differ from the requester.
5 — Governed tools	One Lambda per manifest target, exact-ARN least privilege, private subnets with zero public egress (measured: 0 NAT, 0 IGW). finalize_determination and refer_fraud are absent from the advertised list entirely.
6 — Evidence & regulated data	Per-tenant physical isolation — each tenant has its own tables and its own WORM bucket, not a shared table with a tenant key. Hash-chained append-only ledger with IAM Deny on update and delete. Customer-managed KMS throughout.
7 — Capture & lineage	One account trail: all management events multi-region plus advanced data-event selectors for Bedrock, AgentCore, Lambda and S3. Log-file validation on, delivered to both CloudWatch Logs and a WORM bucket. Reconciliation runs in both directions, so an invoked-but-unaudited tool is an orphan and fails.
8 — Operations & containment	Kill switch (one SSM parameter, IAM-attributed, WORM-audited, 13.9s to effect); per-tenant token and USD budget with reserve-before/commit-after; AWS Budgets ceiling engages the kill switch automatically. The dashboard and alarm surface is the platform's weakest area.
Correct this if you hear it overstated “Every API call in the account” means every call on the governed path; the account-wide net is management events. The trail does not see SageMaker-hosted or self-hosted models, Amazon Q, external model APIs, or uninstrumented workloads (register R4-7, PERIM-7). And “Bedrock over PrivateLink” is not the same claim as “Bedrock runs inside your VPC.”	

7. What we are asking the partner to do

7.1 The one thing that matters, and it is cheap

Independent verification — the headline deliverable

“We have a deny-by-default governance layer for Bedrock agents that has been live-gated end to end on our own account. All of the evidence is ours, which means none of it counts. We want your SA to run the independent verification protocol in a clean account — it is scripted, it takes about half a day, and it either reproduces our results or it does not. If it reproduces, we have joint evidence and a concrete list of what stands between it and a customer pilot. If it does not, you have saved us both a customer conversation.”

That ask is credible because it is falsifiable, cheap for the partner, and it converts our weakest point into their contribution.

7.2 The verification run, step by step

Step	What the partner's SA does	Effort	What it proves
1	Clone the repository at tag v0.7.0-pilot-rc1 and confirm the tag equals main and that the RELEASE file inside the tag names the tag.	10 min	That what they are testing is what we described.
2	Run the offline suite: 510 pass, 4 skip (514 collected) . Confirm the CI security gates are blocking, not report-only.	20 min	The controls have tests, and the tests run.
3	Read FULL-PORTFOLIO-GATE-benefits_runtime_agent.json and the failed run in the same commit side by side — a pass and the failure before it.	20 min	Whether our evidence is honest. This is the cheapest high-signal step.
4	Deploy from zero into a clean AWS account they control , following docs/INDEPENDENT-VERIFICATION.md.	~2 hr	That it deploys somewhere that is not our laptop.
5	Run the full-portfolio gate themselves and record the result — including if it fails.	~1.5 hr	The 21/21, reproduced or refuted, by someone who is not us.
6	Attempt to break it. Suggested order: forge a sanitized_ref; call a tool with a token carrying no custom:tools; call assess_eligibility with deidentified absent; call finalize_determination directly; call Bedrock directly with an IAM user and see whether the bypass alarm fires.	~1 hr	Whether the forbids actually forbid. This is the part we most want them to do.
7	Tear down and confirm zero stack and zero AgentCore residue.	30 min	That it leaves nothing behind in their account.

One prerequisite we owe them

The account must have CloudTrail headroom. The limit is **five trails per region and AWS marks it not adjustable** (L-1568E18E). Each deployed pack needs two. A support case is drafted at docs/ops/CLOUDTRAIL-QUOTA-REQUEST.md and needs a human in the console — the Support API requires a Business or Enterprise plan. Tell them this before they start, not after.

8. Open gaps

Every item below is tracked in WOGplatform/docs/GAP-CLOSURE-BACKLOG.md. Priority is our assessment of what stands between today and a supervised customer pilot on de-identified data.

8.1 Blocking a customer pilot

ID	Gap	Owner	Why it blocks
—	Independent verification	Partner	Everything else in this document is author-produced. This is the only item that changes the credibility of the rest.
—	One real EXTERNAL read-only system-of-record connector, governed end to end	Joint	A governed OAuth2 connector is proven — against a system of record we own. Until one we do NOT own is governed, “it governs your agents” is an inference from our own mock, not a demonstration against a customer’s system.
REL-5	Live re-gates of pharmacovigilance, financial-aid and housing	Us	Three of four packs have never been deployed. The gate harness exists only in benefits.
PAR-2	Housing uplift from 3 of 17 controls	Us	Housing’s CDK is at the July Gate-B level. It should not be shown as a peer of the other three.
PERIM-1b	AWS Organization + SCP tested in a real OU	Customer / partner account team	The only path from detective to preventive account-wide governance. Frequently the deciding question in public sector.
—	Observability uplift — tranche A done, B and C open	Us	Done: named alarms, 3 severity tiers, latency alarms, approval-backlog detection, incident runbook enforced by CI. Open: per-tenant dashboards, synthetic canary, proof that each tier fires live, DR exercise.
R4-15	Enterprise IdP federation round-trip	Us + customer	Production identity has never been exercised.

8.2 Should close, but does not block the conversation

ID	Gap	Status
CHK-1	Checkov baseline burn-down, 488 to a justified residual	Open (P1). Baseline is reviewed and gated so new findings fail CI.
SIG-1	Sign PV / financial-aid / housing manifests; exercise the KMS signer	Open. Benefits uses Ed25519; the others are unsigned.
PERIM-6	Multi-account aggregation to a log-archive account	Open (P1).

ID	Gap	Status
PERIM-7	Non-Bedrock model surfaces — SageMaker endpoints, third-party APIs	Open (P1). Currently outside the capture net, and stated as such.
PERIM-8	Capacity and quota model — reserved concurrency, DynamoDB mode	Open (P2).
L61	CloudTrail trails-per-region quota case	Drafted, not filed. Needs a human in the console.
L37	Teardown does not prune the toolkit ECR repository	Open. Negligible cost; the claim was the problem, and the claim is now precise.
R4-15	Penetration test, DR exercise, SLO, incident-response walkthrough	None performed.

9. Positioning

9.1 What to say

The frame that holds up

A governance control plane that is further along than what a team would build themselves, with a named list of what remains and a scripted protocol for verifying it.

Good for today: architecture workshops; a CISO or AO conversation about what governed agent access should look like; and a supervised pilot on synthetic or de-identified data.

9.2 What not to say

- **Do not call it a product, a solution, or something a customer can deploy against regulated data.** It is none of those and a partner SA will establish that inside an hour. Saying it first costs nothing and buys the rest of the conversation.
- **Do not say “non-bypassable account-wide governance.”** Without an Organization it is detective. Say “non-bypassable *guardrailing*” if the account-level enforced guardrail is applied — that is one control, not a perimeter.
- **Do not conflate audit with observability.** The audit trail is strong and the operations surface is thin. Using the words interchangeably is the fastest way to lose credibility with an SA who then goes and looks.
- **Do not present the four packs as four proven verticals.** Benefits is the reference pack. The other three are the same hash-locked core, not live-proven.

9.3 The recommendation to put in front of the partner

Suggested wording

“This is a platform built primarily for governance, and it is verified as far as one team can verify its own work. If your SA reproduces the gate in a clean account, and we jointly wire one real read-only system-of-record connector and re-gate two more packs, we believe this becomes a full solution a customer can pilot on de-identified data under supervision. We are not asking you to find that customer yet — we are asking you to tell us whether the evidence survives contact with someone who is not us.”

9.4 The sequence

Phase	What happens	Gate to the next phase
A — Verify	Partner SA runs the independent verification protocol in a clean account, including the break-it step.	The gate reproduces, or we learn why it does not.
B — Extend	One real read-only system-of-record connector governed end to end. Live re-gate of a second pack. Observability uplift. Organization + SCP if the partner can supply an OU.	A real system of record is governed, and a second pack has been deployed by someone.
C — Pilot	Named customer. Supervised pilot on de-identified data. Customer's own security engineering, testing and authorization.	Customer's AO accepts the compensating-control statement, or the SCP perimeter is in place.
D —	Not in scope for this conversation. Requires pen test, DR, SLO, IdP	—

Phase	What happens	Gate to the next phase
Production	round-trip, ATO and the customer's formal authorization.	

Appendix — verify any claim in this brief yourself

To check...	Open this
The 21/21 result and every check's raw detail	evidence/FULL-PORTFOLIO-GATE-benefits_runtime_agent.json
That we publish failures too	evidence/FULL-PORTFOLIO-GATE-2026-09-08.json
The thirteen Cedar policies, in full, with their reasoning	policies/*.cedar
Which control maps to which obligation	docs/AUDIT-READINESS.md
Every finding, including the ones where we were wrong	WOGplatform/docs/GAP-CLOSURE-BACKLOG.md (67 rows)
The honesty boundary that governs every other page	WOGplatform/NOT-CLAIMS.md
The reproduction protocol	docs/INDEPENDENT-VERIFICATION.md
What each pack actually wires	WOGplatform/docs/PACK-PARITY.md
The release record and its qualifications	VALIDATED_RELEASE.md, RELEASE-MANIFEST.md
The threat model	docs/THREAT-MODEL.md
The as-deployed architecture	docs/AEGIS-ARCHITECTURE-VERIFIED-2026-09-10.drawio

Final statement of scope

No real or regulated data has ever passed through this platform. All evidence is author-produced on one AWS account and no third party has reproduced it. The platform holds no certification, authorization or ATO of any kind, and nothing in this document is legal or compliance advice. Where a customer's obligations apply, they attach to the AWS services under the shared responsibility model and to the customer's own configuration, operation and authorization.

If any wording anywhere reads as stronger than this, NOT-CLAIMS.md governs.